Why Irradix Never Contains `101`

Exact algorithms, negative bases, and arithmetic of the even shift

Analysis of the `irradix` repository

September 7, 2026

Abstract

The exact integer-quotient version of this repository's golden-ratio encoding has a complete and simple description: its positive codewords are exactly the binary strings beginning with `1` in which every run of zeros between successive ones has even length. Thus `101` is forbidden, as are `10001`, `1000001`, and all longer odd-zero gaps. The proof comes from a contracting, sign-reversing map whose fixed point is φ^{-2}. We derive the Fibonacci count of codewords, an exact integer-only encoder, and the practical consequences for packing. An explicit negative-base correspondence explains the same Fibonacci weights. Residue formulas force prime-free length blocks; a separate theorem gives equidistribution at fixed moduli coprime to 6. A 19-state carry relation constructs exact sums, with a bounded-delay obstruction for canonical streaming output; scaled Fibonacci Horner evaluation constructs products using the adder. Prime counts are compared with a heuristic, not a prime asymptotic. Finite-precision implementations are distinguished from the mathematical encoding throughout.

1 The precise object being proved

Put $\varphi = (1+\sqrt{5})/2$. For a nonnegative integer n, the code's modulo operation, interpreted in exact arithmetic, performs

$$q = \lfloor n/\varphi \rfloor, \qquad d = \lfloor n - \varphi q \rfloor, \qquad \mathcal{E}(n) = \mathcal{E}(q)\, d. \tag{1}$$

Use the empty word for $\mathcal{E}(0)$ while recursing. The public representation of zero is the special string `0`. For positive inputs the recursion terminates because $0 \le q < n$, and $d \in \{0,1\}$ because $0 \le n - \varphi q < \varphi < 2$.

This is *not* the ordinary positional base-φ expansion. For example, the repository represents 2 by `10`, whose ordinary positional value would be φ, not 2. The correct inverse is

$$A(q) = \lceil \varphi q \rceil, \qquad n = A(q) + d. \tag{2}$$

Indeed, since n is an integer, $\lfloor n - \varphi q \rfloor = n - \lceil \varphi q \rceil$. Decoding therefore applies a ceiling at *each* digit. One ceiling around a sum of powers is not equivalent.

Definition 1.1. A digit d is admissible after a prefix with decoded value q if $\lfloor (A(q)+d)/\varphi \rfloor = q$.

Appending `0` is always admissible. For $q > 0$, φq is irrational; write $t(q) = \{\varphi q\} \in (0,1)$. Then

$$\texttt{1} \text{ is admissible after } q \quad \Longleftrightarrow \quad t(q) > 2 - \varphi. \tag{3}$$

To see this, $A(q) = \varphi q + 1 - t(q)$, so the upper quotient bound is $\varphi q + 2 - t(q) < \varphi(q+1)$. The lower quotient bound is automatic. At $q = 0$, the first `1` is admissible and produces $q = 1$.

2 The phase identity: where the golden ratio enters

Set

$$a = \varphi - 1 = \varphi^{-1}, \qquad c = 1 - a = 2 - \varphi = a^2.$$

Thus $0 < a < 1$, $a^2 + a = 1$, and $c \approx 0.381966$. The threshold in (3) is exactly c.

Lemma 2.1 (Effect of appending a zero)**.** *For every positive integer q,*

$$t(A(q)) = a(1 - t(q)).$$

Equivalently, the zero transition $T(t) = a(1 - t)$ satisfies

$$T(t) - c = -a(t - c). \tag{4}$$

Proof. Let $m = \lfloor \varphi q \rfloor$, so $A(q) = m + 1$ and $\varphi q = m + t$. Using $\varphi^2 = \varphi + 1$ gives

$$\varphi A(q) = (m + q + 1) + a(1 - t).$$

The parenthesized expression is an integer and $0 < a(1 - t) < 1$, so the final term is the fractional part. Finally, $a(1 - c) = c$ follows from $a^2 + a = 1$, proving (4). □

Lemma 2.2 (Effect of appending a one)**.** *After any admissible `1`, the phase lies in $[a, 1)$, hence strictly above c.*

Proof. The initial `1` has phase $t(1) = \varphi - 1 = a > c$. For a positive prefix, an admissible `1` means $t > c$. From the previous calculation,

$$\varphi(A(q) + 1) = (m + q + 2) + a(2 - t).$$

Here $a < a(2 - t) < 1$: the upper bound is precisely $t > c$, since $a(2 - c) = 1$. Thus the new fractional part is $a(2 - t)$. □

Theorem 2.3 (The even-gap characterization)**.** *A nonempty binary word is $\mathcal{E}(n)$ for some positive integer n if and only if it starts with `1` and every zero run between successive `1`s has even length. Trailing zeros may have either parity.*

Proof. Immediately after a `1`, let the phase be $t_0 > c$. After j zeros, repeated use of (4) gives

$$t_j - c = (-a)^j (t_0 - c). \tag{5}$$

Because $a > 0$, this is positive for even j and negative for odd j. By (3), the next `1` is admissible exactly when j is even. This proves necessity.

Conversely, start with `1` and read a word satisfying the stated condition. Every zero is admissible. Before each subsequent one the number of zeros since the previous one is even, so (5) makes that one admissible as well. Each inverse step has exactly the quotient and remainder in (1); reversing the steps therefore recovers the word from its decoded integer. This proves sufficiency. No further digit needs to follow trailing zeros, so they have no parity restriction. □

Corollary 2.4. *For every $j \geq 0$, no exact codeword contains $1\,0^{2j+1}\,1$. In particular, `101` never occurs.*

A short version of the proof. A `1` puts the phase above c; a `0` reflects it below c; a following `1` would require it to be above c. That contradiction excludes `101`. Equation (4) is the exact reason $\varphi^2 = \varphi + 1$ matters. Irrationality alone does not imply this property: for base $\sqrt{2}$, the same quotient algorithm maps 4 to `101`. We do not claim here that φ is the only real base with this exclusion.

3 Fibonacci counts and an exact arithmetic replacement

The permitted suffixes after the initial `1` are precisely

$$(\mathtt{1} \mid \mathtt{00})^*(\varepsilon \mid \mathtt{0}).$$

Equivalently, remember whether the zero count since the last one is even or odd. From even, a one stays even and a zero goes odd; from odd, only a zero is permitted, returning to even. This is the classical *even shift* language [1, 3, 4]. The equivalence between the repository's quotient algorithm and that language was established above; the language itself is not new.

Let $F_0 = 0$, $F_1 = 1$, and $F_{j+2} = F_{j+1} + F_j$. There are exactly F_{k+1} positive codewords of length k. Indeed, if C_r counts suffixes of length r from the even state, then $C_0 = 1$, $C_1 = 2$, and $C_r = C_{r-1} + C_{r-2}$. Consequently

$$|\mathcal{E}(n)| = k \quad \Longleftrightarrow \quad F_{k+2} - 1 \le n \le F_{k+3} - 2. \tag{6}$$

Here is an independent verification that these counts occur in integer order. The quotient $\lfloor n/\varphi \rfloor$ is nondecreasing in n; each quotient has either a zero child or a zero child followed by a one child. Induction therefore shows that the binary values of $\mathcal{E}(n)$ are strictly increasing. Summing the length counts yields (6).

Length k	Smallest n	Largest n	Codewords
1	1	1	1
2	2	3	2
3	4	6	3
4	7	11	5
5	12	19	8
6	20	32	13

3.1 A direct Fibonacci formula

For a valid word $w = 1d_{k-2} \cdots d_0$ of length k,

$$\boxed{D(w) = F_{k+2} - 1 + \sum_{j=0}^{k-2} d_j F_{j+1}.} \tag{7}$$

To prove this, rank the length-k words lexicographically, starting at rank zero. Whenever a one is chosen with r positions remaining, the skipped zero branch contains exactly F_r completions. At an odd state a zero is forced and skips nothing. Thus the rank is the sum in (7); add the smallest integer of that length. For example, `1001` decodes to $F_6 - 1 + F_1 = 7 + 1 = 8$.

Encoding reverses this ranking procedure: locate k using (6), subtract $F_{k+2} - 1$, and at each even state choose zero if the remaining rank is below F_r, otherwise choose one and subtract F_r. At an odd state output the forced zero. This uses $O(k)$ integer additions, comparisons and subtractions, with $O(k)$ Fibonacci table entries. This is an arithmetic-operation count; large-integer operations are not constant-time.

3.2 An independent square-root reference

An exact version of the original algorithm is also available from integer square roots:

$$\lfloor n/\varphi \rfloor = \frac{\operatorname{isqrt}(5n^2) - n}{2} \text{ rounded down,}$$

$$A(0) = 0, \qquad A(q) = \left\lfloor \frac{q + \operatorname{isqrt}(5q^2)}{2} \right\rfloor + 1 \quad (q > 0).$$

These formulas follow from $\varphi = (1 + \sqrt{5})/2$ and the fact that $\sqrt{5}\,q$ is irrational for positive integer q. They provide an independent reference against which the faster Fibonacci implementation can be tested, without choosing floating-point precision.

4 Consequences for packing

The expansion is structural. The Fibonacci counts imply

$$|\mathcal{E}(n)| = \log_\varphi n + O(1) = 1.440420\ldots \log_2 n + O(1).$$

This is not compression of unrestricted integers. Its redundancy buys a constrained bit language. The even-shift transition matrix $\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ has largest eigenvalue φ, explaining the same constant combinatorially.

If only `101` must be forbidden, a denser code is possible. Allowing all `101`-free words discards the extra odd-gap restrictions. Its three-state prefix automaton has characteristic polynomial $\lambda^3 - 2\lambda^2 + \lambda - 1$ and largest root $\lambda \approx 1.754878$. Enumerative block coding can approach $1/\log_2 \lambda \approx 1.232$ output bits per input bit before framing. This is a different code and requires a separate boundary design.

Delimiters still need boundary handling. The absence of `101` inside each payload does not alone prove that joining payloads with `101` is safe: a payload ending in `10` can create an earlier crossing occurrence. The repository uses doubling and a suffix repair. A simple alternative prototype is

$$\mathcal{E}(n+1)\ \texttt{00}\ \texttt{101}.$$

The two guard zeros prevent a premature crossing delimiter; decoding splits at `101`, removes exactly two zeros, validates the payload, and subtracts one. It costs five framing bits per value, including a terminal delimiter. It is a new format, not a compatible replacement. Neither this delimiter nor the even-gap rule is a checksum.

Length-first packing remains the useful large-integer variant. Writing raw binary payloads and encoding only their lengths changes the per-value cost to $B + O(\log B)$ for a B-bit integer. That explains the repository's large-integer advantage over VByte; direct Irradix still pays approximately $1.44B$. Elias delta and Fibonacci coding are relevant established baselines [5, 6]. For a deterministic sample of 1,000 integers of 50–100 decimal digits, the reproduced storage counts were:

Method	Stored bits, including byte rounding
Raw payload only (not a framed code)	248,161
Direct Irradix	361,216
Length-first Irradix	264,240
Elias delta, applied to $n+1$	263,072
VByte	286,984

These are sample-dependent sizes, not an entropy bound or a throughput comparison. The repository baseline is ordinary VByte, not the Stream VByte implementation studied in [7]. If many lengths repeat, block widths or compressed length runs are further opportunities.

5 The negative-golden-ratio correspondence

The even shift predates this project [1, 3]. More specifically, Frougny and Lai [2], Examples 2–3, explicitly identify the $(-\varphi)$-shift with the even shift. We now relate the repository's integer construction to that negative-base expansion. This is a derivation for this map, not a claim of novelty priority.

Put $a = \varphi - 1$, $c = 2 - \varphi$, and, for $n \geq 1$, define

$$x(n) = c - \{\varphi n\} = (\lfloor \varphi n \rfloor + 2) - (n+1)\varphi \in (-a, c).$$

The greedy negative-base map on $[-a, c)$ has digit and remainder

$$D_-(x) = \lfloor -\varphi x + a \rfloor, \qquad T_-(x) = -\varphi x - D_-(x).$$

Theorem 5.1 (An exact negative-base identity)**.** *If $\mathcal{E}(n) = d_{k-1} \cdots d_0$, the greedy fractional expansion of $x(n)$ in base $-\varphi$ is*

$$\boxed{x(n) = (.\, d_0 d_1 \cdots d_{k-1} 1\, 000 \cdots)_{-\varphi}.}$$

Proof. For an admissible step $n = \lceil \varphi q \rceil + d$ with $q \geq 1$, the phase identity gives $t(n) = a(1 + d - t(q))$. Since $a(1-c) = c$, this becomes

$$x(n) = -\frac{x(q) + d}{\varphi}.$$

Because $0 < x(q) + a < 1$, the next greedy digit is d and the remainder is exactly $x(q)$. Thus the map reads the Irradix word from right to left until it reaches $n = 1$. The boundary calculation is

$$3 - 2\varphi \xrightarrow{1} -a \xrightarrow{1} 0 \xrightarrow{0} 0.$$

The first digit here is the leading one of $\mathcal{E}(n)$; the second is the extra terminal one in the stated expansion. This also explains why the recurrence must not be extended naively through $n = 0$. □

For example, $\mathcal{E}(2) = \texttt{10}$ and $x(2) = 5 - 3\varphi = (.011)_{-\varphi}$. The positional expansion represents $x(n)$, not n.

One quadratic algebra, two roles. Let $\psi = 1 - \varphi = -1/\varphi$. The identity reads

$$x(n) = \sum_{j=0}^{k-1} d_j \psi^{j+1} + \psi^{k+1}.$$

Induction gives $\psi^r = F_{r+1} - F_r\varphi$. Equating coefficients of φ in the irrational basis $1, \varphi$ yields

$$n = F_{k+1} - 1 + \sum_{j=0}^{k-1} d_j F_{j+1},$$

which is (7) because $d_{k-1} = 1$. The graph matrix $\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ has eigenvalues φ and ψ: one controls growth of the language, the other is the contraction and reflection of its arithmetic phase.

6 Residues, prime-free blocks, and a corrected heuristic

Write $B(n)$ for the word $\mathcal{E}(n)$ interpreted as an ordinary binary integer. Primality here refers to $B(n)$, independently of primality of n.

Theorem 6.1 (Residue determined by length and trailing zeros)**.** *For a canonical word of length k with t trailing zeros,*

$$B(n) \equiv \begin{cases} 0 & \text{if } k - t \text{ is even,} \\ (-1)^t & \text{if } k - t \text{ is odd} \end{cases} \pmod 3.$$

Proof. Let r be the number of ones. Successive one-bit exponents differ by an odd number, since the intervening zero runs have even length. Their parities alternate, and $2^j \equiv (-1)^j \pmod 3$. The contributions therefore cancel in pairs; if r is odd, the remaining term is $(-1)^t$. The highest and lowest one-bit exponents are $k-1$ and t, so $r - 1 \equiv k - 1 - t \pmod 2$, proving the statement. □

Corollary 6.2 (Prime-free blocks)**.** *Every prime image $B(n) > 3$ has odd binary length and is* $1 \pmod 6$*. For every even $k \geq 4$, all inputs in*

$$[F_{k+2} - 1, F_{k+3} - 2]$$

map to composite integers.

Proof. An even image above 2 is composite. An odd image has $t = 0$, so at even length it is divisible by 3, and at odd length it is $1 \pmod 3$. Use (6) to obtain the input intervals. □

Examples of these intervals are 7–11, 20–32, 54–87, 143–231, 376–608, and 986–1595.

The candidate proportion has no limit. There are F_{k-1} words ending in one at each length $k \geq 2$: their final edge is the one-loop from the even state, and the number of paths ending at that state after $k - 2$ steps is F_{k-1}. By the residue theorem, the count coprime to 6 is zero at even lengths and F_{k-1} at odd lengths $k \geq 3$. Include the length-one image 1, which is coprime to 6 but not prime. For $N_k = F_{k+3} - 2$, summation gives

$$C_k = \#\{n \leq N_k : \gcd(B(n), 6) = 1\} = \begin{cases} F_k & k \text{ odd}, \\ F_{k-1} & k \text{ even}. \end{cases}$$

The summation identity follows by telescoping $F_{2j} = F_{2j+1} - F_{2j-1}$. Hence C_k/N_k tends to φ^{-3} along odd lengths and φ^{-4} along even lengths, approximately 23.61% and 14.59%. These distinct subsequential limits prove nonexistence of a density over all input prefixes. This concerns coprimality to 6, not a prime asymptotic.

6.1 Uniformity away from the local obstruction

Theorem 6.3 (Fixed-modulus equidistribution)**.** *Fix a positive integer m coprime to 6. Among canonical words of length k, each binary residue modulo m has proportion tending to $1/m$. Convergence is exponential for each fixed m.*

Proof. Attach a residue to each zero-parity state. The transitions are

$$(E, r) \xrightarrow{1} (E, 2r + 1), \quad (E, r) \xrightarrow{0} (O, 2r), \quad (O, r) \xrightarrow{0} (E, 2r),$$

with all arithmetic modulo m. The case $m = 1$ is immediate. Otherwise, let L be the multiplicative order of 4 modulo m. The two E-to-E blocks `00` and `11` act as $r \mapsto 4r$ and $r \mapsto 4r + 3$. Use mL blocks. Their initial coefficient is 1 modulo m. At the m positions having $0, L, \ldots, (m-1)L$ blocks remaining, choose either block; choose `00` elsewhere. Selecting `11` at j of those positions gives $r + 3j$. For $j = 0, \ldots, m - 1$ these cover all residues, since 3 is invertible. All E states communicate; invertibility of 2 connects every O state to them. The one-loop at $(E, -1)$ makes the graph aperiodic.

Its adjacency matrix is thus primitive. Positive left and right eigenvectors are constant across residues, with state weights $(\varphi, 1)$, and direct multiplication gives eigenvalue φ. This uses that $r \mapsto 2r$ and $r \mapsto 2r + 1$ are permutations. Perron–Frobenius convergence from the initial state $(E, 1)$ therefore gives equal limiting total weight to each residue. The remaining eigenvalues have modulus strictly smaller than φ, which gives exponential convergence with constants depending on m. □

The same reasoning, restricted to the penultimate E state, proves uniformity conditional on a final one. This rules out persistent individual residue biases at each fixed prime at least 5. It provides no uniform error estimate as m grows, so it does not prove a prime counting theorem.

6.2 Measurements and what remains conjectural

For $N \geq 3$, a natural local heuristic is

$$H(N) = 2 + \sum_{\substack{n \leq N,\ B(n) > 3 \\ B(n) \equiv 1 \pmod 6}} \frac{3}{\log B(n)}.$$

The 2 counts the exceptional primes 2 and 3. The factor 3 is the ordinary prime-density factor for the residue class 1 modulo 6, not a fitted parameter. Transferring this density to the restricted set is heuristic.

Inputs 1 through N	Exact mapped primes	$H(N)$
100	12	14.32
1,000	94	94.46
10,000	466	464.53
100,000	4,319	4,342.72

These nested samples are not independent trials. The checked-in data also include complete length blocks, including the plateau of 94 primes from $N = 985$ through 1595. Dividing prime density by a magnitude expansion factor, as in the historical discussion, is unjustified; the logarithmic scale and residue restrictions both matter. Neither these experiments nor fixed-modulus uniformity establish infinitely many prime images. Maynard's restricted-decimal-digit theorem [8] concerns different hypotheses and is not a theorem for this language.

7 Finite-state integer arithmetic

The negative-base correspondence now gives a working digit adder and multiplier, as well as a direct incrementer and decrementer. The implementation `research/phi_arithmetic.py` supports signed canonical words and uses neither whole-value integer decoding nor Fibonacci tables. Earlier finite-state and multipass Fibonacci arithmetic is substantial prior art [9, 10]; the construction below handles this repository's particular even-shift representation.

7.1 Successor and predecessor

To increment a positive word, scan after its leading one and locate the rightmost zero read from the even state. Replace it by one and replace the entire following suffix by zeros. If there is no such position, the input is all ones; return one followed by as many zeros as the old word had digits. Treat $\mathtt{0} \mapsto \mathtt{1}$ separately.

Theorem 7.1 (Successor correctness)**.** *This algorithm maps $\mathcal{E}(n)$ to $\mathcal{E}(n+1)$ for every nonnegative integer n.*

Proof. The earlier order argument identifies integer order with shortlex order on the canonical language. A zero can change to one exactly in the even state. Choosing its rightmost possible occurrence selects the next lexicographic branch; all zeros give that branch's least continuation. If no such zero exists, there cannot be any zero, since the first would have been in an even state. The input is therefore the last word of its length, and its successor is the first word of the next length. □

The predecessor changes the rightmost nonleading one to zero, then fills the remaining positions with their greatest legal continuation: one forced zero, if space remains, followed by ones. A word consisting of a one and only zeros moves to all ones of the previous length, with $\mathtt{1} \mapsto \mathtt{0}$. The reverse lexicographic argument proves correctness. Signed operations apply these rules to the magnitude, reversing increment and decrement for negative inputs. All require $O(k)$ digit work for an input of length k.

7.2 Augmented Fibonacci weights and a finite carry bound

For $w = \mathcal{E}(n)$ prefix an extra one and write $U(n) = \mathtt{1}w$. If $W(d_k \cdots d_0) = \sum_j d_j F_{j+1}$, the decoder gives

$$W(U(n)) = n + 1.$$

This includes $U(0) = \mathtt{10}$, with weight $F_2 = 1$, and is unchanged by leading-zero padding. Consequently, for three aligned augmented words,

$$a + b = c \quad \Longleftrightarrow \quad W(U(a) + U(b) - U(c)) = 1,$$

where word addition and subtraction mean digitwise operations. Each difference digit e belongs to $\{-1, 0, 1, 2\}$.

Evaluate the difference polynomial by Horner's rule in $\mathbb{Z}[\varphi]$. For $C = u + v\varphi$, the next difference digit gives

$$C \longmapsto \varphi C + e, \qquad \boxed{(u, v) \longmapsto (v + e, u + v).}$$

Start at $(0, 0)$. The sum of the coefficients of φ^j is F_{j+1}, including the constant case $j = 0$. Thus the final sum $u + v$ is exactly W, and the acceptance condition is $u + v = 1$, not zero.

Theorem 7.2 (A finite rectangle contains every successful carry path)**.** *Every prefix of a successful difference word has* $-5 \le u \le 5$ *and* $-4 \le v \le 4$*.*

Proof. Set $\psi = 1 - \varphi = -1/\varphi$ and $C' = u + v\psi$. For every prefix,

$$|C'| \le 2 \sum_{j \ge 0} |\psi|^j = \frac{2}{1 - 1/\varphi} < 6.$$

At an accepting endpoint, $u + v = 1$, hence $C' = 1 - \varphi v$. This confines integer v to $-3, \ldots, 4$ and gives $|C| = |1 + v/\varphi| < 4$. If a prefix has r digits remaining, unwinding the remaining Horner steps yields

$$|C_{\text{prefix}}| < 4\varphi^{-r} + \frac{2(1 - \varphi^{-r})}{\varphi - 1} \le 4.$$

Since $2/(\varphi - 1) < 4$, the bound holds for every prefix. Finally

$$v = \frac{C - C'}{\sqrt{5}}, \qquad u = \frac{\varphi C' - \psi C}{\sqrt{5}}$$

give $|v| < 5$ and $|u| < 6$, proving the integer bounds. □

Generate transitions inside this 99-state rectangle. There are 67 states reachable from the start. Removing states with no path to acceptance leaves **19 carry states**, with terminals $(-1, 2), (0, 1), (1, 0), (2, -1)$. These finite counts are reproduced by the implementation; no minimality claim is made. Reachability and backward trimming preserve every accepting path.

7.3 Constructing the output, not merely recognizing a sum

Given a pair of input digits, try both output bits and combine the carry machine with the five-state automaton for padded augmented outputs

$$\mathtt{0}^*(\mathtt{10} \mid \mathtt{11}(\mathtt{1} \mid \mathtt{00})^*(\varepsilon \mid \mathtt{0})).$$

Its states distinguish padding, the augmentation one, the special zero word, and the positive even/odd states. The product has at most 95 states.

The solver retains reachable product states for each column, storing one predecessor and output bit per state. At the end it selects an accepting state and recovers the word by tracing the path backward. Merging paths at a state loses no possible completion: both paths have the same carry and language state, and future constraints depend only on those states.

Theorem 7.3 (Adder soundness and completeness)**.** *The solver returns the unique canonical representation of the sum.*

Proof. An accepting path has a canonical augmented output and satisfies the carry equation, proving soundness. Conversely, the canonical sum exists. If the longer input has k digits, the sum has at most $k + 2$ digits:

$$a + b \le 2(F_{k+3} - 2) < F_{k+5} - 1,$$

where the right side is the first value of length $k + 3$; zero operands cause no difficulty. Using $k + 3$ columns including augmentation therefore fits the correct output. Its carries lie in the proved rectangle and survive trimming, establishing completeness. The encoding's bijection gives uniqueness. □

For subtraction $c = a - b \geq 0$, use differences $b_i + c_i - a_i$ in the same machine, solving $c + b = a$. Signed addition compares magnitudes by shortlex order, chooses addition or subtraction, and applies the sign.

Time and path-history space are $O(k)$, with bounded small-integer carry operations. Exhausting the reachable subset graph gives 41 frontier subsets for addition and 39 for subtraction, each of size at most five. Thus the implementation has at most five active candidate states per column, even on arbitrary input bit pairs. This is a checked finite enumeration, not a Lean theorem. A separate unsigned relation checker can validate supplied operands and result by a single deterministic carry pass without storing path history.

7.4 Multiplication by scaled Fibonacci Horner evaluation

The multiplier uses the carry adder as a building block. For nonnegative operands a, b, scan the augmented multiplier $U(b)$ from most significant first, maintaining two values represented as Irradix words:

$$(x, y) = (0, 0), \qquad (x, y) \longmapsto (x + y + da, x)$$

for each multiplier digit d. At the end return $x - a$. The implementation starts after the prefixed augmentation one, at $(a, 0)$, and chooses the shorter operand as the multiplier.

Theorem 7.4 (Multiplier correctness)**.** *Scaled Fibonacci Horner evaluation returns $\mathcal{E}(ab)$ when its operations are carried out by the exact digit adder and subtractor.*

Proof. For a processed prefix with digits d_j indexed from the right, the invariant is

$$x = a \sum_j d_j F_{j+1}, \qquad y = a \sum_j d_j F_j.$$

Appending a digit shifts the old Fibonacci indices up by one. The recurrence $F_{j+2} = F_{j+1} + F_j$ therefore gives $x + y + da$ as the new first value and x as the new second value. After the entire augmented word, $x = aW(U(b)) = a(b + 1)$; the final subtraction gives ab. □

Zero and one have explicit shortcuts. Signs are handled by multiplying magnitudes and applying the exclusive-or of operand signs. For input lengths k, l, the number of iterations is $\min(k, l)$ and intermediate words have $O(k + l)$ digits. Thus the implementation takes $O(\min(k, l)(k + l))$ digit work and $O(k + l)$ working space. It is quadratic for equally sized operands; the full multiplier is not claimed to be a finite-state transducer.

This is the Fibonacci-weight analogue of shift-and-add. It never decodes the operands into their whole integer values. Tests include 16,384 unsigned and 4,225 signed pairs, random products, distributivity, 64 Fibonacci boundaries, a product of two 1,000-digit words, and a 20,000-digit word multiplied by 6. An independent unbounded weight oracle checks the long results. Decode/multiply/re-encode remains substantially faster in the measured samples; details are recorded in `research/multiplication_measurements.json`.

7.5 An obstruction to bounded-delay output

Theorem 7.5 (No deterministic bounded-delay one-way incrementer)**.** *When input and canonical output are read in the same direction, neither most-significant-first nor least-significant-first incrementing can emit successive digits with a fixed lookahead independent of input length.*

Proof. For most-significant-first reading, compare $\mathtt{1}^k$ with $\mathtt{1}^{k-1}\mathtt{0}$. They share $k-1$ initial digits, but their increments are $\mathtt{10}^k$ and $\mathtt{1}^k$, whose second digits differ. With a common padded output width, the first digits differ. For least-significant-first reading, compare $\mathtt{10}^{k-1}$ and $\mathtt{110}^{k-2}$, for $k \geq 3$. They share $k-2$ initial digits in this direction. An input ending in $t > 0$ zeros increments to an odd binary image exactly when t is odd. The two trailing-zero counts have opposite parity, so the first output digits differ. In both cases the common input prefix can be arbitrarily long, even when the common input length is known. □

A general adder inherits this obstruction by fixing one operand to one. This excludes neither multipass or two-way machines nor the offline finite-state solver above. Online results for other Fibonacci normal forms [9] concern a different canonical output language.

Measurements. The arithmetic checks include 100,001 successor/predecessor pairs, 65,536 unsigned and 16,641 signed addition pairs, 1,000 random signed pairs up to 4,096 bits, and 300 Fibonacci boundaries. A 100,000-digit addition is checked against an independent unbounded Fibonacci-weight oracle. The prototype demonstrates finite carries and linear digit work; in the recorded small-size benchmarks the generic carry solver remains slower than decoding, adding with Python integers, and re-encoding. The direct incrementer is faster on those sampled workloads. Machine and workload details are recorded in `research/arithmetic_measurements.json`.

8 Generalization and concrete further questions

The construction in Section 7 resolves the finite-state arithmetic question for this representation. The original fractional coordinate identity remains informative:

$$x(n+m) = x(n) + x(m) - c + \lfloor \{\varphi n\} + \{\varphi m\} \rfloor .$$

Its correction is zero or one. The augmented-word construction accounts for this arithmetic without computing fractional coordinates. Improving the adder's table representation and storage is a concrete engineering target; quantitative spectral bounds as the residue modulus grows remain a separate target toward a sieve for prime images.

A graph eigenvalue alone is insufficient. Consider zero gaps divisible by 3. Its graph has return paths of lengths 1 and 3, so the growth rate β satisfies $\beta^3 = \beta^2 + 1$. The positive root lies between $4/3$ and $3/2$. Repeated floor quotient encoding gives quotients $4 \to 2 \to 1 \to 0$ and digits $1, 0, 1$ from least to most significant: $\mathcal{E}_\beta(4) = \mathtt{101}$. This violates the proposed gap rule. Thus merely matching the graph's growth rate does not reproduce its language by this arithmetic construction.

A limited uniqueness statement. The golden ratio is the only quadratic Pisot number in $(1, 2)$. Indeed, let β be such a number and $\gamma \in (-1, 1)$ its other conjugate. The integer trace is 1 or 2. Trace 2 makes the integer norm $\beta(2-\beta)$ strictly between 0 and 1, impossible. Trace 1 makes $\beta(1-\beta)$ strictly between -2 and 0, so the norm is -1 and $\beta^2 - \beta - 1 = 0$. This singles out the quadratic contraction in the binary range; it does not classify all irrational quotient languages.

9 Verification and scope

The mathematical theorem concerns *exact* quotient and ceiling operations. The repository's fixed-precision `mpmath` code and its C++ floating-point version are approximations. The reproduced run finds round-trip failures near 10^{100} at the configured precision of 100 decimal digits. Such numerical failures do not contradict the theorem, but they matter for an implementation claiming arbitrary-size integers.

The accompanying Python reference independently checks the original integer-square-root construction and the Fibonacci construction on 100,001 values; checks 500 Fibonacci length thresholds; and tests 4,369 short sequences per experimental codec. The machine-checked Lean companion formalizes the exact ceiling-step model, the phase transition argument, and the forbidden odd-gap result. Its build instructions and precise theorem inventory are in `research/lean/README.md`. The negative-base identity and residue theorems added here are written proofs, not part of the Lean formalization. Separate exact Python checks cover 10,000 negative-base expansions and 100,000 modular identities. A residue automaton checks counting formulas through length 100 and is compared with direct enumeration through length 13 for eight moduli. The prime experiment uses a sieve through 6,692,032 for 100,000 inputs. The new `PhiArithmetic.lean` companion separately verifies the integer carry invariant and proves the acceptance condition equivalent to addition, assuming the augmented words have the prescribed Fibonacci weights. It does not formalize the finite rectangle bound, graph trimming, canonical augmentation identity, incrementer, or output recovery. It also verifies scaled Fibonacci Horner evaluation and the final subtraction that converts the augmented weight to the product. Those integer invariants do not formally verify the Python multiplier. The Python implementation and the packing formats themselves are not formally verified by that Lean proof. Numerical tests are supplementary evidence, not a substitute for the all-integers theorem.

References

[1] D. Lind and B. Marcus, *An Introduction to Symbolic Dynamics and Coding*, Cambridge University Press (1995). https://sites.math.washington.edu/SymbolicDynamics/.

[2] C. Frougny and A. C. Lai, *On Negative Bases*, DLT 2009, LNCS 5583, pp. 252–263 (2009). doi:10.1007/978-3-642-02737-6_20. https://www.irif.fr/~cf/publications/dlt-final.pdf.

[3] M. Pivato and R. Yassawi, *Asymptotic Randomization of Sofic Shifts by Linear Cellular Automata*, preprint (2003; revised 2006). https://arxiv.org/abs/math/0306136.

[4] D. Glasscock, J. Moreira, and F. K. Richter, *Additive and geometric transversality of fractal sets in the integers*, Journal of the London Mathematical Society (2024). doi:10.1112/jlms.12902. The even shift is discussed as a binary digit restriction.

[5] A. Apostolico and A. S. Fraenkel, *Robust Transmission of Unbounded Strings Using Fibonacci Representations*, Purdue technical report 85-545 (1985). https://docs.lib.purdue.edu/cstech/464/.

[6] D. A. Lelewer and D. S. Hirschberg, *Data Compression*, section 3, universal codes. https://ics.uci.edu/~dhirschb/pubs/DC-Sec3.html.

[7] D. Lemire, N. Kurz, and C. Rupp, *Stream VByte: Faster Byte-Oriented Integer Compression*, preprint (2017). https://arxiv.org/abs/1709.08990.

[8] J. Maynard, *Primes with restricted digits*, preprint (2016). https://arxiv.org/abs/1604.01041.

[9] C. Frougny, *On-line finite automata for addition in some numeration systems*, RAIRO–Theoretical Informatics and Applications 33(1), 79–101 (1999). https://www.numdam.org/item/ITA_1999__33_1_79_0.pdf.

[10] C. Ahlbach, J. Usatine, C. Frougny, and N. Pippenger, *Efficient Algorithms for Zeckendorf Arithmetic*, Fibonacci Quarterly 51(3), 249–255 (2013). https://www.fq.math.ca/Papers1/51-3/AhlbachUsatineFrougnyPippenger.pdf.